

HW0

Welcome to CS4644/7643! This notebook aims to provide a self-check for your numpy/torch knowledge while teaching you some of the development tools you will use throughout the semester.

Important: Follow the exercises in order - some depend on previous steps!

Google Colab

If you are using Google Colab, configure and run the cell below to mount your Google Drive and set up the correct working directory for the assignment. You may ignore this cell if you are working locally.

```
In [6]: try:
        from google.colab import drive
        import os
        drive.mount('/content/drive')
        FOLDERNAME = "hw0_student_version"
        FULL_PATH = "/content/drive/MyDrive/" + FOLDERNAME
        assert os.path.exists(FULL_PATH)
    except Exception:
        import os
        FULL_PATH = None
        if os.path.exists("requirements.txt"):
            FULL_PATH = os.getcwd()
        else:
            for root, dirs, files in os.walk('.'):
                dirs[:] = [d for d in dirs if not d.startswith('.') and d not in
                           ['hw0_student_version']]
                if 'hw0_student_version' in dirs:
                    FULL_PATH = os.path.abspath(os.path.join(root, 'hw0_student_
                    break
            if not FULL_PATH:
                FULL_PATH = os.getcwd()
```

```
In [7]: %cd -q $FULL_PATH
```

```
In [9]: !uv -q pip install -r requirements.txt
```

Local Setup

If you are running locally, follow the instructions in `hw0_instructions.pdf` to set up your environment with uv.

```
In [10]: !uv sync
```

Resolved **92 packages** in 23ms
Checked **72 packages** in 15ms

Exercises

Complete the functions in `main.py` file.

You can run tests with the cell below or in the terminal.

```
In [11]: # this cell is intended to help you track your progress as you complete the  
!python -m pytest "tests/test_numpy.py" -v
```

```

===== test session starts =====
=====
platform darwin -- Python 3.13.7, pytest-8.4.1, pluggy-1.6.0 -- /Users/aashu
tosh/anaconda3/envs/general_env/bin/python
cachedir: .pytest_cache
rootdir: /Users/aashutosh/Documents/Academics - GaTech/Fall 2026/DL/hw0/hw0_
student_version
configfile: pyproject.toml
plugins: anyio-4.12.0, jaxtyping-0.3.2
collected 14 items

tests/test_numpy.py::TestBasicArrayOperations::test_create_checkerboard PASS
ED [ 7%]
tests/test_numpy.py::TestBasicArrayOperations::test_create_diagonal_matrix P
ASSED [ 14%]
tests/test_numpy.py::TestBasicArrayOperations::test_create_zero_vector PASSE
D [ 21%]
tests/test_numpy.py::TestBasicArrayOperations::test_place_random_ones PASSED
[ 28%]
tests/test_numpy.py::TestImageProcessing::test_channel_last_to_first PASSED
[ 35%]
tests/test_numpy.py::TestImageProcessing::test_rgb_to_bgr PASSED [ 4
2%]
tests/test_numpy.py::TestArrayManipulation::test_broadcast_multiply_rows PAS
SED [ 50%]
tests/test_numpy.py::TestArrayManipulation::test_convert_dtype PASSED [ 5
7%]
tests/test_numpy.py::TestArrayManipulation::test_negate_range_inplace PASSED
[ 64%]
tests/test_numpy.py::TestArrayManipulation::test_one_hot_encode PASSED [ 7
1%]
tests/test_numpy.py::TestArrayManipulation::test_pad_with_zeros PASSED [ 7
8%]
tests/test_numpy.py::TestArrayManipulation::test_sort_by_column PASSED [ 8
5%]
tests/test_numpy.py::TestArrayManipulation::test_subtract_row_means PASSED [
92%]
tests/test_numpy.py::TestGradientComparison::test_gradient_consistency PASSE
D [100%]

===== 14 passed in 0.86s =====
=====

```

```

In [12]: # you can run individual tests like this
!python -m pytest "tests/test_numpy.py::TestBasicArrayOperations::test_creat

```

```

===== test session starts =====
=====
platform darwin -- Python 3.13.7, pytest-8.4.1, pluggy-1.6.0 -- /Users/aashu
tosh/anaconda3/envs/general_env/bin/python
cachedir: .pytest_cache
rootdir: /Users/aashutosh/Documents/Academics - GaTech/Fall 2026/DL/hw0/hw0_
student_version
configfile: pyproject.toml
plugins: anyio-4.12.0, jaxtyping-0.3.2
collected 1 item

tests/test_numpy.py::TestBasicArrayOperations::test_create_zero_vector PASSE
D [100%]

===== 1 passed in 0.63s =====
=====

```

14. Numerical and Analytical Differentiation

Background

In deep learning and optimization, we often need to compute derivatives of complex functions that involve vectors and matrices. While automatic differentiation is commonly used in modern frameworks, understanding numerical differentiation is crucial for grasping the underlying concepts and debugging.

One simple method for numerical differentiation is the finite difference method. This method approximates the derivative of a function by calculating the rate of change over a small interval.

For a function $f(\mathbf{x})$ where $\mathbf{x}$ is a vector, the i -th partial derivative can be approximated using the central difference formula:

$$\frac{\partial f}{\partial x_i} \approx \frac{f(\mathbf{x} + h\mathbf{e}_i) - f(\mathbf{x} - h\mathbf{e}_i)}{2h}$$

Where:

- $f(\mathbf{x})$ is the function we want to differentiate
- $\mathbf{x}$ is the input vector
- h is a small step size
- $\mathbf{e}_i$ is the i -th standard basis vector (a vector with 1 in the i -th position and 0 elsewhere)

Your task is to implement a function that can numerically differentiate an arbitrary Python function that takes a vector input, using the central difference method. You'll then apply this to various functions involving vectors and matrices.

14.a Implement Numerical Differentiation for Vector Inputs

Write a function `numerical_gradient(func, x, h=1e-5)` that takes:

- `func` : A Python function that takes a numpy array as input and returns a scalar
- `x` : The point at which to evaluate the gradient (a numpy array)
- `h` : The step size (default to 1e-5)

The function should return the approximate gradient of `func` at `x` as a numpy array.

Make sure to use vectorized operation to make your computation efficient.

14.b Differentiating Simple Functions

Apply your `numerical_gradient` function to the following functions and compare the results with their analytical gradients:

- $f(\mathbf{x}) = \mathbf{x}^T \mathbf{x}$
- $f(\mathbf{x}) = \sin(x_1) + \cos(x_2) + \cos(x_3)$
- $f(\mathbf{x}) = \mathbf{x}^T A \mathbf{x}$, where A is a 3x3 matrix with random entries

For each function, calculate the numerical gradient at $\mathbf{x} = [1, 1, 1]$ functions. Compare these to the true gradients.

```
In [13]: import numpy as np
from main import numerical_gradient, f1, f2, f3, analytical_gradient_f1, and

# Test point (now 3D for all functions)
x_3d = np.array([1.0, 1.0, 1.0])

# Random 3x3 matrix for function 3
A = np.random.randn(3, 3)

# Compute and compare gradients
def compare_gradients(f, analytical_grad, x, *args):
    numerical_grad = numerical_gradient(lambda x: f(x, *args), x)
    analytical_grad = analytical_grad(x, *args)

    print(f"Numerical gradient: {numerical_grad}")
    print(f"Analytical gradient: {analytical_grad}")
    print(f"Difference: {np.linalg.norm(numerical_grad - analytical_grad)}")
    print()

# Test function 1
print("Function 1: f(x) = x^T x")
compare_gradients(f1, analytical_gradient_f1, x_3d)

# Test function 2
print("Function 2: f(x) = sin(x_1) + cos(x_2) + cos(x_3)")
compare_gradients(f2, analytical_gradient_f2, x_3d)

# Test function 3
```

```
print("Function 3: f(x) = x^T A x")
compare_gradients(f3, analytical_gradient_f3, x_3d, A)
```

Function 1: $f(x) = x^T x$
 Numerical gradient: [2. 2. 2.]
 Analytical gradient: [2. 2. 2.]
 Difference: 2.2694036439624267e-11

Function 2: $f(x) = \sin(x_1) + \cos(x_2) + \cos(x_3)$
 Numerical gradient: [0.54030231 -0.84147098 -0.84147098]
 Analytical gradient: [0.54030231 -0.84147098 -0.84147098]
 Difference: 2.8592656090443453e-11

Function 3: $f(x) = x^T A x$
 Numerical gradient: [2.61603194 -0.20578642 0.71049715]
 Analytical gradient: [2.61603194 -0.20578642 0.71049715]
 Difference: 1.260584803321442e-11

14.c: Differentiate a Complex Vector Function

Consider the following more complex function:

$$f(\mathbf{x}) = (\mathbf{x}^T A \mathbf{x}) * \sin(\mathbf{x}^T \mathbf{b}) + e^{-\mathbf{x}^T \mathbf{x}}$$

Where:

- $\mathbf{x}$ is an n -dimensional vector (use $n = 5$ for this assignment)
- A is an $n \times n$ matrix
- $\mathbf{b}$ is an n -dimensional vector

Implement this function in Python, generating random values for A and $\mathbf{b}$. Use your `numerical_gradient` function to compute its gradient at $\mathbf{x} = [1, 1, 1, 1, 1]$.

```
In [14]: import numpy as np
from main import numerical_gradient, complex_function

# Generate random A and b
n = 5
np.random.seed(42) # for reproducibility
A = np.random.randn(n, n)
b = np.random.randn(n)

# Compute gradient at x = [1, 1, 1, 1, 1]
x = np.ones(n)
numerical_grad = numerical_gradient(lambda x: complex_function(x, A, b), x)

print(f"Gradient at x = {x}:")
print(numerical_grad)
```

Gradient at x = [1. 1. 1. 1. 1.]:
 [-3.0171028 -1.86919823 2.25118 6.7941431 4.03785221]

14.d: Compare with Analytical Gradient Computed by PyTorch

PyTorch is a popular open-source machine learning library that we'll be using throughout this semester. It offers dynamic computational graphs, which allow for flexible model design, and provides automatic differentiation capabilities for efficient gradient computations. PyTorch can leverage GPU to massively accelerate training and inference complex models. One of its key features is the ability to compute analytical gradients automatically, which is crucial for training deep neural networks.

For a gentle introduction to PyTorch, let's compare the analytical gradient computed by PyTorch with the numerical gradient we implemented earlier. If you're running this notebook locally, make sure you have PyTorch (`torch`) installed. If you're using Google Colab, PyTorch should already be available.

```
In [15]: import torch
from main import numerical_gradient, complex_function

# Generate random A and b
n = 5
np.random.seed(42) # for reproducibility
A = np.random.randn(n, n)
b = np.random.randn(n)

# Compute gradient at x = [1, 1, 1, 1, 1]
x = np.ones(n)

# Complex vector function (PyTorch version)
def complex_function_torch(x, A, b):
    return (x @ A @ x) * torch.sin(x @ b) + torch.exp(-x @ x)

# Function to compute analytical gradient using PyTorch
def compute_pytorch_gradient(x_np, A_np, b_np):
    x = torch.tensor(x_np, requires_grad=True, dtype=torch.float32)
    A = torch.tensor(A_np, dtype=torch.float32)
    b = torch.tensor(b_np, dtype=torch.float32)

    y = complex_function_torch(x, A, b)
    # compute gradient is as simple as calling .backward() on the output of
    y.backward()

    return x.grad.numpy()

# Let's compare gradients!
analytical_grad = compute_pytorch_gradient(x, A, b)
numerical_grad = numerical_gradient(lambda x: complex_function(x, A, b), x)

print("Analytical gradient (PyTorch):")
print(analytical_grad)
print("\nNumerical gradient (Finite Difference):")
print(numerical_grad)
```

```
print("\nDifference (L2 norm):")
print(np.linalg.norm(analytical_grad - numerical_grad))
```

Analytical gradient (PyTorch):

```
[-3.0171027 -1.8691981  2.2511802  6.7941427  4.0378523]
```

Numerical gradient (Finite Difference):

```
[-3.0171028 -1.86919823  2.25118      6.7941431  4.03785221]
```

Difference (L2 norm):

```
4.512653216965659e-07
```

Submitting your code

```
In [ ]: # on colab, you may need to install chromium to convert the notebook to pdf
# !playwright install-deps chromium
# !playwright install chromium
```

```
In [ ]: !python collect_submission.py
```

```
=== HW0 Submission Collection Script ===
```

```
Converting notebook to PDF...
```

```
In [ ]:
```